

Lab 4: Laboratório de Pinger ICMP

Neste laboratório, você obterá uma melhor compreensão do Protocolo de Mensagens de Controle da Internet (ICMP). Você aprenderá a implementar uma aplicação Ping usando mensagens de solicitação e resposta ICMP.

Ping é uma aplicação de rede usada para testar se um host específico é acessível em uma rede IP. Também é usada para testar a interface de rede do computador ou como um teste de latência. Ela funciona enviando pacotes de "echo reply" ICMP para o host de destino e aguardando as respostas ICMP de "echo reply". O "echo reply" é às vezes chamado de pong. O Ping mede o tempo de ida e volta, registra a perda de pacotes e imprime um resumo estatístico dos pacotes de resposta recebidos (o tempo mínimo, máximo e médio de ida e volta e, em algumas versões, o desvio padrão da média).

Sua tarefa é desenvolver sua própria aplicação Ping em Python. Sua aplicação usará ICMP, mas, para mantê-la simples, não seguirá exatamente a especificação oficial do RFC 1739. Observe que você só precisa escrever o lado do cliente do programa, já que a funcionalidade necessária no lado do servidor está embutida na maioria dos sistemas operacionais.

Você deve completar a aplicação Ping para que envie solicitações de ping a um host especificado com intervalos de aproximadamente um segundo. Cada mensagem contém uma carga útil de dados que inclui um carimbo de tempo. Após enviar cada pacote, a aplicação espera até um segundo para receber uma resposta. Se passar um segundo sem resposta do servidor, o cliente assume que o pacote de ping ou o pacote de pong foi perdido na rede (ou que o servidor está inativo).

Código

Abaixo, você encontrará o código esqueleto para o cliente. Você deve completar o código esqueleto. Os locais onde você precisa preencher o código estão marcados com `#Fill in start` e `#Fill in end`. Cada local pode exigir uma ou mais linhas de código.

Notas adicionais

1. No método "receiveOnePing", você precisa receber a estrutura ICMP_ECHO_REPLY e buscar as informações necessárias, como checksum, número de sequência, tempo de vida (TTL), etc. Estude o método "sendOnePing" antes de tentar completar o método "receiveOnePing".
2. Você não precisa se preocupar com o checksum, pois ele já está dado no código.
3. Este laboratório exige o uso de `raw sockets`. Em alguns sistemas operacionais, pode ser necessário ter privilégios de administrador/root para rodar o programa Pinger.
4. Veja o final deste exercício de programação para mais informações sobre ICMP.

Testando o Pinger

Primeiro, teste seu cliente enviando pacotes para localhost, ou seja, 127.0.0.1. Depois, você deve ver como sua aplicação Pinger se comunica pela rede, fazendo ping a servidores em diferentes continentes.

Apresentação

Em sua apresentação, você deve mostrar e explicar o código completo do cliente e mostrar o ping para quatro hosts de destino, cada um em um continente diferente.

Esta atividade vale nota. A implementação básica resulta em nota máxima 6,0.

Exercícios opcionais

1. Atualmente, o programa calcula o tempo de ida e volta (RTT) para cada pacote e o imprime individualmente. Modifique isso para corresponder à forma como o programa de ping padrão funciona. Você precisará relatar o RTT mínimo, máximo e médio ao final de todos os pings do cliente. Além disso, calcule a taxa de perda de pacotes (em porcentagem).
2. Seu programa só pode detectar timeouts ao receber respostas ICMP de eco. Modifique o programa Pinger para interpretar os códigos de erro de resposta ICMP e exibir os resultados de erro correspondentes para o usuário. Exemplos de códigos de erro de resposta ICMP são: 0 - Rede de Destino Inalcançável, 1 - Host de Destino Inalcançável.

Código

```
from socket import *
import os
import sys
import struct
import time
import select
import binascii

ICMP_ECHO_REQUEST = 8

def checksum(string):
    csum = 0
    countTo = (len(string) // 2) * 2
    count = 0

    while count < countTo:
        thisVal = ord(string[count+1]) * 256 + ord(string[count])
        csum = csum + thisVal
        csum = csum & 0xffffffff
        count = count + 2

    if countTo < len(string):
        csum = csum + ord(string[len(string) - 1])
        csum = csum & 0xffffffff

    csum = (csum >> 16) + (csum & 0xffff)
    csum = csum + (csum >> 16)
    answer = ~csum
    answer = answer & 0xffff
    answer = answer >> 8 | (answer << 8 & 0xff00)
    return answer

def receiveOnePing(mySocket, ID, timeout, destAddr):
    timeLeft = timeout

    while 1:
        startedSelect = time.time()
        whatReady = select.select([mySocket], [], [], timeLeft)
        howLongInSelect = (time.time() - startedSelect)
        if whatReady[0] == []: # Timeout
            return "Request timed out."

        timeReceived = time.time()
        recPacket, addr = mySocket.recvfrom(1024)

        #Fill in start

        # Fetch the ICMP header from the IP packet

        #Fill in end

        timeLeft = timeLeft - howLongInSelect
        if timeLeft <= 0:
            return "Request timed out."

def sendOnePing(mySocket, destAddr, ID):
    # Header is type (8), code (8), checksum (16), id (16), sequence (16)
    myChecksum = 0

    # Make a dummy header with a 0 checksum
```

```

header = struct.pack("bbHHh", ICMP_ECHO_REQUEST, 0, myChecksum, ID, 1)
data = struct.pack("d", time.time())

# Calculate the checksum on the data and the dummy header
myChecksum = checksum(str(header + data))

# Get the right checksum, and put in the header
if sys.platform == 'darwin':
    myChecksum = htons(myChecksum) & 0xffff
else:
    myChecksum = htons(myChecksum)

header = struct.pack("bbHHh", ICMP_ECHO_REQUEST, 0, myChecksum, ID, 1)
packet = header + data

mySocket.sendto(packet, (destAddr, 1)) # AF_INET address must be tuple, not str

def doOnePing(destAddr, timeout):
    icmp = getprotobyname("icmp")
    mySocket = socket(AF_INET, SOCK_RAW, icmp)

    myID = os.getpid() & 0xFFFF # Return the current process i
    sendOnePing(mySocket, destAddr, myID)
    delay = receiveOnePing(mySocket, myID, timeout, destAddr)

    mySocket.close()
    return delay

def ping(host, timeout=1):
    dest = gethostbyname(host)
    print("Pinging " + dest + " using Python:")
    print("")

    # Send ping requests to a server separated by approximately one second
    while 1:
        delay = doOnePing(dest, timeout)
        print(delay)
        time.sleep(1)

```